Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

Лабораторная работа №3

«Функциональные возможности языка Python Часть 1»

ДИСЦИПЛИНА: «Парадигмы и конструкции языков программирования»

Выполнил: студент гр. ИУ5-32Б _____ (Сердюков А.С.)
(Подпись) (Ф.И.О.)

Проверил: _____ (Гапанюк Ю.Е.)
(Подпись) (Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:
- Оценка:

2026 г.

Цель и задание

Реализовать генераторы, итератор с удалением повторов и сортировку с функцией ключа. Работа содержит задачи 1–4 общего задания №3–4.

Задача 1 — генератор field для выбора полей словарей с пропуском None. Задача 2 — gen_random для целых чисел в замкнутом диапазоне. Задача 3 — итератор Unique с параметром ignore_case, заданным через kwargs. Задача 4 — два однострочных варианта сортировки по убыванию модуля, с lambda и без неё.

Описание решения

field выдаёт значение одного поля или словарь нескольких непустых полей; отсутствующие поля рассматриваются как None. Значения 0, False и пустая строка сохраняются. gen_random использует randint с включёнными границами. Unique лениво читает исходный итератор, сравнивает нормализованные ключи и возвращает исходный элемент. Для строк применён casefold.

Unique поддерживает список, генератор и нехешируемые элементы. Список просмотренных ключей позволяет сравнивать даже вложенные списки; худшая сложность — $O(n^2)$, память $O(u)$, где u — число уникальных значений. Сортировка использует sorted с key=abs или key=lambda value: abs(value), reverse=True. При равных модулях сохраняется исходный порядок.

Порядок запуска

Python 3.10 или новее. Из корня репозитория создать виртуальное окружение и установить зависимости командой python -m pip install -r requirements.txt.

```
python lab_03/lab_python_fp/field.py
python lab_03/lab_python_fp/gen_random.py
python lab_03/lab_python_fp/unique.py
python lab_03/lab_python_fp/sort.py
```

Результаты выполнения

```
$ python lab_03/lab_python_fp/field.py
['Ковер', 'Диван для отдыха']
[{'title': 'Ковер', 'price': 2000}, {'title': 'Диван для отдыха'}]

$ python lab_03/lab_python_fp/gen_random.py
[1, 1, 1, 3, 2]

$ python lab_03/lab_python_fp/unique.py
['a', 'A', 'b', 'B']
['a', 'b']

$ python lab_03/lab_python_fp/sort.py
[123, 100, -100, -30, 30, 4, -4, 1, -1, 0]
[123, 100, -100, -30, 30, 4, -4, 1, -1, 0]

Итог тестового запуска (полный журнал tests.txt)
$ python -m pytest lab_03 -v --tb=short
3 passed in 0.02s
```

Рисунок 1 — Вывод фактического запуска программы

Экранная форма сформирована из сохранённого stdout. Полные журналы: results/demo.txt и results/tests.txt.

Проверка и выводы

Для этой работы успешно выполнено проверок: 3. Общий набор всех шести работ содержит 61 тест; полный прогон завершился без ошибок. Версии среды и журналы сохранены в validation/ и results/tests.txt.

Все четыре задачи выполнены в отдельных запускаемых файлах пакета lab_python_fp. Полученные инструменты используются в конвейере следующей работы. ignore_case=True объединяет разные регистры согласно основному тексту и примерам задания.

Источник задания

https://github.com/ugapanyuk/BKIT_2022/wiki/lab_python_fp

Текст программы

```
# Файл lab_python_fp/__init__.py

"""Общий пакет лабораторных 3 и 4 (PEP 420 path extension)."""
from pkgutil import extend_path

__path__ = extend_path(__path__, __name__)


# Файл lab_python_fp/field.py

"""Задача 1. Ленивое извлечение непустых полей."""


def field(items, *args):
    if not args:
        raise ValueError("Укажите хотя бы одно поле")
    for item in items:
        if len(args) == 1:
            value = item.get(args[0])
            if value is not None:
                yield value
        else:
            values = {key: item[key] for key in args if item.get(key) is not
                None}
            if values:
                yield values


if __name__ == "__main__":
    goods = [{"title": "Ковер", "price": 2000},
             {"title": "Диван для отдыха", "price": None}, {}]
    print(list(field(goods, "title")))
    print(list(field(goods, "title", "price")))


# Файл lab_python_fp/gen_random.py

"""Задача 2. Генератор целых чисел с включёнными границами."""
import random


def gen_random(num_count, begin, end):
    if num_count < 0 or begin > end:
        raise ValueError("Нужны неотрицательное количество и begin <= end")
    for _ in range(num_count):
        yield random.randint(begin, end)


if __name__ == "__main__":
    random.seed(22)
    print(list(gen_random(5, 1, 3)))


# Файл lab_python_fp/unique.py

"""Задача 3. Итератор, сохраняющий порядок и исходные значения."""
```

Текст программы Продолжение

```
class Unique:
    def __init__(self, items, **kwargs):
        self.items = iter(items)
        self.ignore_case = kwargs.pop("ignore_case", False)
        if kwargs or not isinstance(self.ignore_case, bool):
            raise TypeError("Допустим только bool-параметр ignore_case")
        self.seen = []

    def __iter__(self):
        return self

    def __next__(self):
        for item in self.items:
            key = item.casefold() if self.ignore_case and isinstance(item,
                str) else item
            if key not in self.seen:
                self.seen.append(key)
                return item
        raise StopIteration


if __name__ == "__main__":
    data = ["a", "A", "b", "B", "a"]
    print(list(Unique(data)))
    print(list(Unique(iter(data), ignore_case=True)))


# Файл lab_python_fp/sort.py

"""Задача 4. Два однострочных способа сортировки."""
data = [4, -30, 30, 100, -100, 123, 1, 0, -1, -4]


if __name__ == "__main__":
    print(sorted(data, key=abs, reverse=True))
    print(sorted(data, key=lambda value: abs(value), reverse=True))


# Файл test_generators.py

import pytest
from lab_python_fp.field import field
from lab_python_fp.unique import Unique
from lab_python_fp.gen_random import gen_random


def test_field_omits_only_none():
    data = [{"a": 0}, {"a": None}, {}, {"a": False}, {"a": ""}]
    assert list(field(data, "a")) == [0, False, ""]
    assert list(field(data, "a", "b")) == [{"a": 0}, {"a": False}, {"a": ""}]
    with pytest.raises(ValueError):
        list(field(data))
```

Текст программы Продолжение

```
def test_unique_generator_preserves_values():
    assert list(Unique(iter(["A", "a", "Б"]), ignore_case=True)) == ["A", "Б"]
    assert list(Unique(["A", "a"])) == ["A", "a"]
    assert list(Unique([[1], [1], [2]])) == [[1], [2]]


def test_random_bounds_and_empty():
    assert list(gen_random(3, 7, 7)) == [7, 7, 7]
    assert list(gen_random(0, 1, 2)) == []
    values = list(gen_random(100, -2, 2))
    assert len(values) == 100 and all(-2 <= n <= 2 for n in values)
    with pytest.raises(ValueError):
        list(gen_random(-1, 0, 2))
```